

Environments and workflows (Conda, Nextflow)

Bioinformatics 101 for colon model experiments



Petra Polakovicova
September 2026

What we'll cover

1. The big picture – why do we need to manage environments and workflows?
2. Conda
3. Nextflow
4. Quick demonstration (In terminal)

What we'll cover

1. The big picture – why do we need to manage environments and workflows?
2. Conda
3. Nextflow
4. Quick demonstration (In terminal)

Lecture goal: to give you a practical grasp of managing environments (Conda) and workflows (Nextflow) — so you understand how they make computational analysis FAIR, and why adopting them is worth it.

What we'll cover

- 1. The big picture – why do we need to manage environments and workflows?**
2. Conda
3. Nextflow
4. Quick demonstration (In terminal)

The big picture

Two tools, two halves of one problem



CONDA

Manages ENVIRONMENTS

“How do I get the exact right software installed, without it fighting other software?”

- Isolated collections of tools + versions
- Solves “dependency hell”
- Fixes “works on my machine”



NEXTFLOW

Manages WORKFLOWS

“How do I chain many steps into an automated pipeline that runs anywhere, at any scale?”

- Orchestrates steps & data flow
- Parallelism + crash-safe restart
- Portable: laptop → HPC → cloud

The big picture

Three hells of a bioinformatician

1

Dependency hell

- Tool A needs numpy 1.19.
- Tool B needs 1.26.
- Install both globally and something breaks.
- Now imagine this multiplied across dozens of tools.

2

“Works on my machine” hell

- Runs on your laptop.
- Fails on a colleague’s or the cluster.
- Results that can’t be reproduced aren’t science.

3

Scaling hell

- download → QC → trim → denoise → tax. assign → summarize
- 100s of samples
- Process crashes mid-run.
- Bash scripts don’t cut it.

Conda fixes #1 and #2

Nextflow fixes #3 (and, with Conda/containers, #2 too)

What we'll cover

1. The big picture – why do we need to manage environments and workflows?
- 2. Conda**
3. Nextflow
4. Quick demonstration (In terminal)

Conda: Environments & package management

What Conda actually is

1

Package manager

Installs, removes, updates software of any language (Python, R, Java, C libs, CLI bio-tools, CUDA).

2

Environment manager

Creates & isolates those collections so projects don't collide.

Conda: Environments & package management

What Conda actually is

1

Package manager

Installs, removes, updates software of any language (Python, R, Java, C libs, CLI bio-tools, CUDA).

2

Environment manager

Creates & isolates those collections so projects don't collide.

The solver

- When you ask for a package, Conda doesn't just download it - it **solves** a constraint problem:
 - Find a set of versions for *everything* (the package, its dependencies, their dependencies...) that are mutually compatible with what you already have.
- This is genuinely a satisfiability (SAT) problem, which is why complex environments can take a while to solve.

Conda: Environments & package management

An environment is just a folder

- A **Conda environment** is a self-contained folder that holds a specific Python (or R, or other) plus a specific set of packages at specific versions.
- **Activating** an environment just saying your computer: use the tools and versions that are stored here.
- Not a VM. Not a container. Not magic.

```
# activating environment
(base) povp:~$ conda activate wmg
(wmg) povp:~$
```

Conda: Environments & package management

How to install conda

Installer	What it is	Note
Anaconda Distribution	Huge bundle: Conda + 250+ pkgs + GUI	Large; commercial “defaults” channel
Miniconda	Minimal: Conda + Python essentials	From Anaconda Inc.; still points at defaults
Miniforge	Minimal: Conda + mamba, conda-forge preset	Recommended · BSD/MIT · independent

Conda: Environments & package management

How to install conda

Installer	What it is	Note
Anaconda Distribution	Huge bundle: Conda + 250+ pkgs + GUI	Large; commercial “defaults” channel
Miniconda	Minimal: Conda + Python essentials	From Anaconda Inc.; still points at defaults
Miniforge	Minimal: Conda + mamba, conda-forge preset	Recommended · BSD/MIT · independent

- <https://docs.conda.io/projects/conda/en/latest/user-guide/install/index.html>
- <https://docs.conda.io/projects/conda/en/latest/user-guide/getting-started.html>

Conda: Environments & package management

How to install conda

Installer	What it is	Note
Anaconda Distribution	Huge bundle: Conda + 250+ pkgs + GUI	Large; commercial “defaults” channel
Miniconda	Minimal: Conda + Python essentials	From Anaconda Inc.; still points at defaults
Miniforge	Minimal: Conda + mamba, conda-forge preset	Recommended · BSD/MIT · independent



Licensing wrinkle: Anaconda Inc.’s ToS can require a paid license for larger orgs using their installers/defaults channel. Universities & companies have moved to Miniforge + conda-forge (free, community-run).

Conda: Environments & package management

Channels (where packages come from)

A **channel** is a repository of Conda packages.

1

conda-forge

big community channel; the de-facto standard

<https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-channels.html>

2

bioconda

bioinformatics tools; depends on conda-forge

3

defaults

Anaconda Inc.'s channel (licensing caveat)

Conda: Environments & package management

The core command vocabulary

```
● ● ●  
  
# create an env with specific packages  
$ conda create -n wmgts python=3.11 kraken2 bracken bowtie2 samtools  
  
# activate = use this env's software  
$ conda activate microbiome  
$ conda deactivate  
  
# add / remove into an env  
$ conda install -n microbiome multiqc metaphlan humann  
$ conda remove -n microbiome multiqc  
  
# inspect  
$ conda env list      # all environments  
$ conda list          # packages in active env  
  
# update  
$ conda update kraken2  
$ conda update -all  
  
# delete an env (it's just a folder)  
$ conda env remove -n microbiome
```

Conda: Environments & package management

environment.yml

Typing `conda create` by hand isn't reproducible. The proper artifact is an environment file, usually `environment.yml`:

```
# environment.yml
name: microbiome
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.11
  - kraken2=2.1.3
  - bracken=2.9
  - metaphlan=4.1.1
  - bowtie2=2.5.4
  - samtools=1.19
  - pip:
    - some-pypi-only-pkg==2.0.1
```

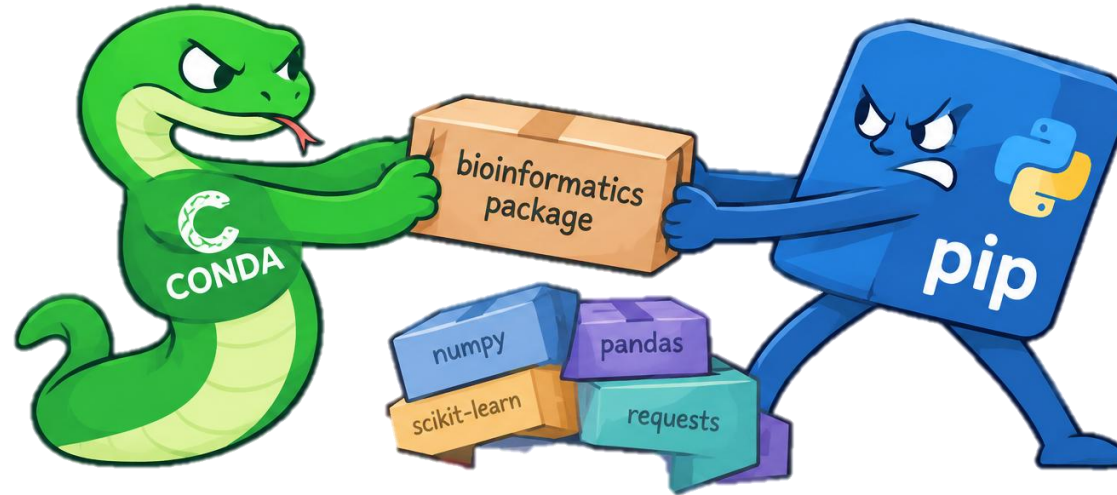
```
# build the exact same environment
$ conda env create -f environment.yml

# capture what you have to yaml
$ conda env export > environment.yml

# capture what you explicitly asked for
$ conda env export --from-history
```

Conda: Environments & package management

Conda vs pip



- **Conda** installs packages of any language from Conda channels
- It solves compatibility across all of its packages
- **pip** installs **Python** packages from PyPI
- It does not manage non-Python dependencies

Conda: Environments & package management

Conda vs pip

- They can coexist, but order matters:
 - install everything you can with **Conda** first,
 - then **pip install** the remainder into the activated Conda env.



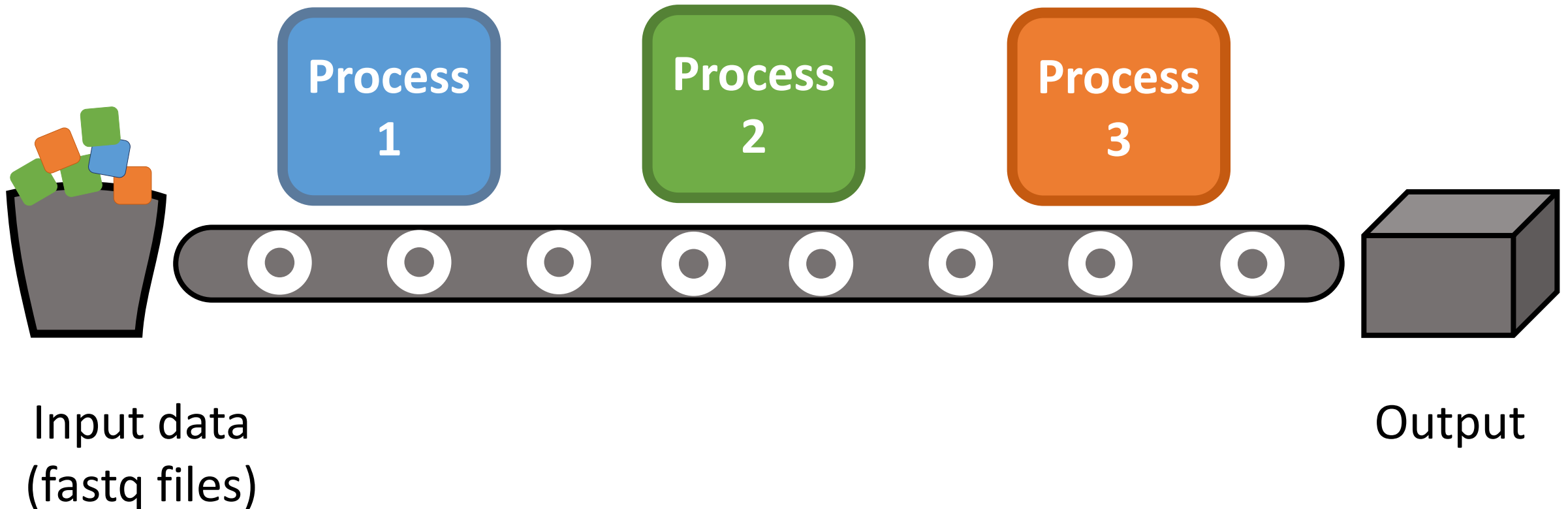
Possible issue: Installing pip packages first (or mixing repeatedly) can lead to Conda "not knowing" what pip changed, producing broken environments.

What we'll cover

1. The big picture – why do we need to manage environments and workflows?
2. Conda
- 3. Nextflow**
4. Quick demonstration (In terminal)

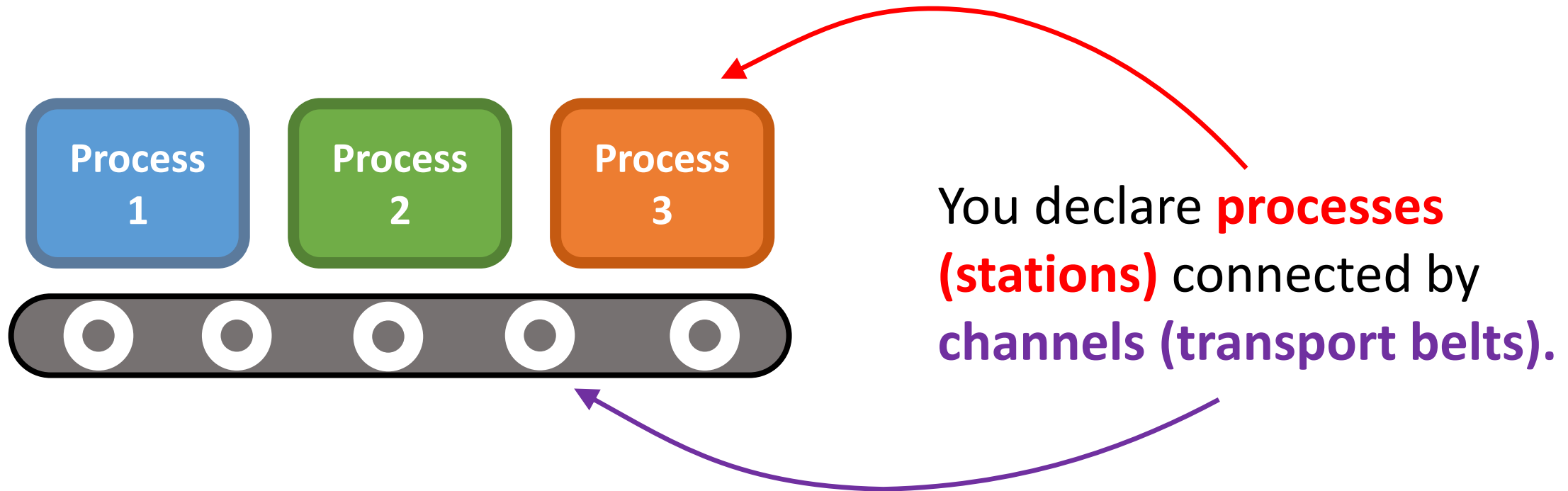
Nextflow – workflow orchestration

Dataflow: an assembly line, not a script



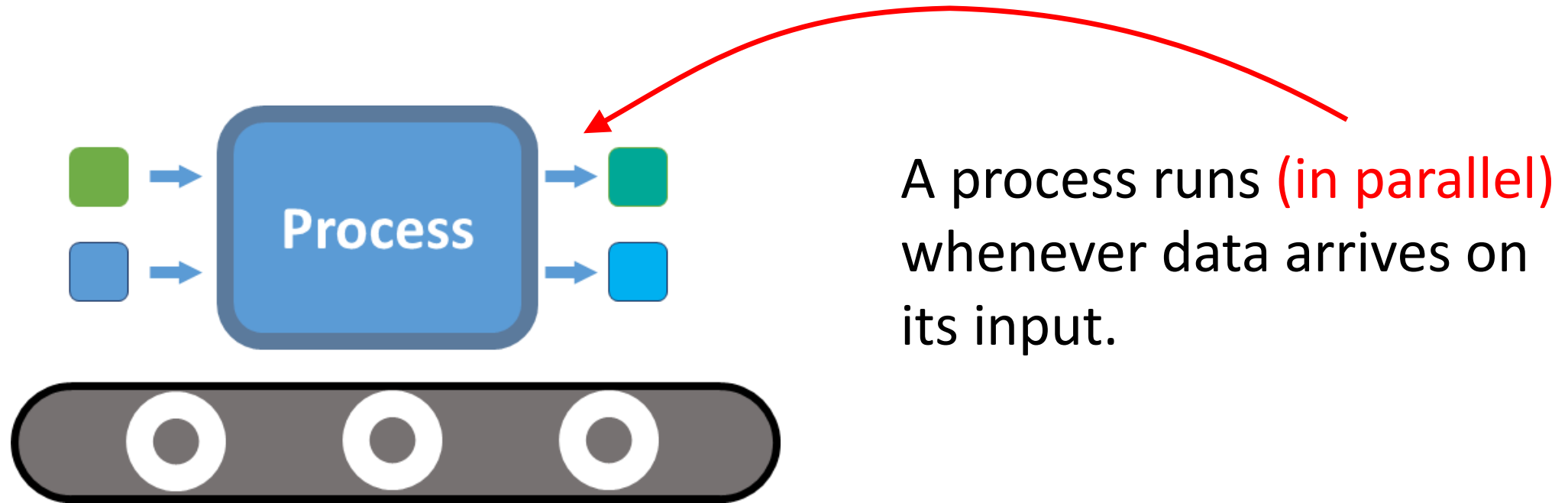
Nextflow – workflow orchestration

Dataflow: an assembly line, not a script



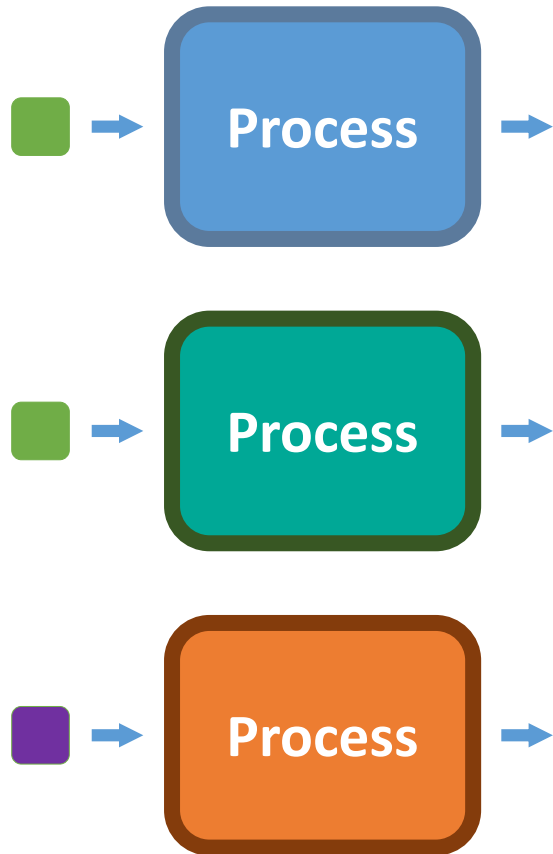
Nextflow – workflow orchestration

Dataflow: an assembly line, not a script



Nextflow – workflow orchestration

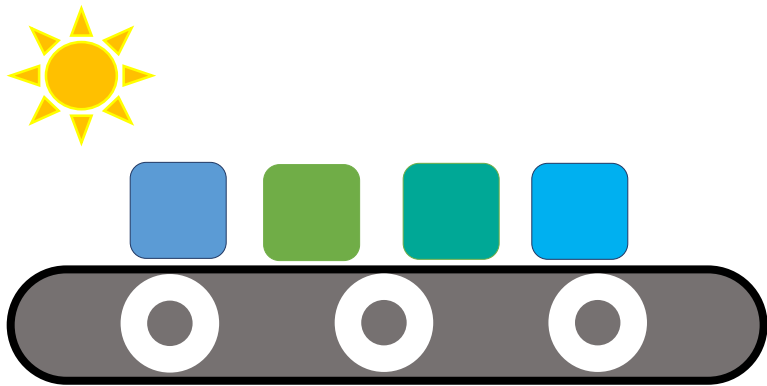
Dataflow: an assembly line, not a script



- You don't tell each station when to work.
- A station works whenever an item arrives on its belt.
- Many identical items can be worked on at different stations simultaneously

Nextflow – workflow orchestration

Dataflow: an assembly line, not a script



The line runs itself:

- You wire the belts -> the line schedules itself (no manual ordering)
- Each task is isolated
- Parallelism (200 items on a channel -> up to 200 runs at once, no loops)

Nextflow – workflow orchestration

The four building blocks

process

a step

channel

data stream

operator

transforms

workflow

the wiring

Nextflow – workflow orchestration

Process (a step)

process

a step

channel

data stream

operator

transforms

workflow

the wiring

```
process TRIM {
    input:
    tuple val(sample_id), path(reads)

    output:
    tuple val(sample_id), path("*.trimmed.fastq.gz")

    script:
    """
    cutadapt ${reads}
    """
}
```

input:

what the process consumes from a channel

output:

what it emits — must match files the script makes

script:

the real shell command (the actual tool call)

Nextflow – workflow orchestration

channel - the stream that feeds steps

process

a step

channel

data stream

operator

transforms

workflow

the wiring

```
Channel.fromPath('data/*.fastq.gz')    // one item per file
Channel.fromFilePairs('*_{R1,R2}.fastq.gz') // paired reads -> tuples
```

Nextflow – workflow orchestration

operator - reshape the stream

process

a step

channel

data stream

operator

transforms

workflow

the wiring

```
reads_ch
  .map { f -> tuple(f.simpleName, f) }
  .filter { id, f -> id.startsWith('sample') }
  .set { tidy_reads }
```

map

transform each item

filter

keep some items

collect

gather all → one list

groupTuple

group items by a key

join

merge two channels by key

view

print items (debug)

Nextflow – workflow orchestration

worklow – wire everything into a pipeline

process

a step

channel

data stream

operator

transforms

workflow

the wiring

```
workflow {  
  reads = Channel.fromFilePairs(params.reads)  
  QC(reads)  
  TRIM(reads)  
  TAX_ASSIGNMENT(TRIM.out)  
}
```

reads



QC



TRIM



TAX_ASSIGNMENT

Nextflow – workflow orchestration

–resume: never redo finished work

- Each task runs in its own dir, which Nextflow remembers.
- Re-run with -resume and it skips unchanged tasks, re-running only what's new or downstream.

COUNT_READS (sampleA)

CACHED

COUNT_READS (sampleB)

RE-RUN

COUNT_READS (sampleC)

CACHED

GC (sampleB)

RE-RUN

What we'll cover

1. The big picture – why do we need to manage environments and workflows?
2. Conda
3. Nextflow
- 4. Quick demonstration (In terminal)**

Quick demonstration

- <https://training.nextflow.io/latest/envsetup/>

Take home messages

- **Conda** = the right software, kept separate. Each project gets its own environment (just a folder), so tools and versions never clash.
- **Nextflow** = your analysis as an automatic assembly line. Steps run in parallel on any number of samples, anywhere, and -resume never redoes finished work.
- **Together** = reproducible (FAIR) science. Share your environment.yml and your pipeline, and anyone can get the same results.

Thank you for your attention!

Questions?



Petra Polakovicova
September 2026